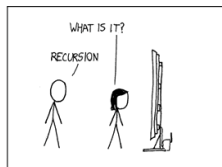
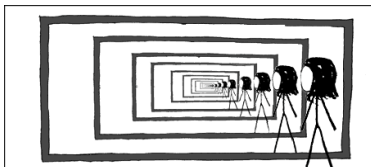


CS319: Scientific Computing

Functions and Quadrature

Dr Niall Madden

Week 4: 5th and 7th, February, 2025



Slides and examples: <https://www.niallmadden.ie/2425-CS319>

1 Overview of this week's classes

- Why quadrature?

2 Functions

- Header
- Function definition
- A mathematical function
- E.g, Prime?
- void functions

3 Numerical Integration

- The basic idea
- The code
- Trapezium Rule as a function

4 Functions as arguments to functions

5 Functions with default arguments

6 Pass-by-value

7 Exercises

Slides and examples:

<https://www.niallmadden.ie/2425-CS319>

Overview of this week's classes

This week, we will study the use of **functions** in C++, which we started (very briefly) at the end of Week 3.

In Scientific Computing, we use the term “**function**” in two different, but related ways:

- ▶ A mathematical function, such as $f(x) = e^{-x}$ or $u(x, y) = \sin(\pi x) \cos y$.
- ▶ A function we code to perform a task, such as determining if an integer is positive. Or, often, working with mathematical functions: to calculate derivatives at a point (Lab 2) or integrals.

And often we'll combine both ideas!

Overview of this week's classes

However, we'll motivate some of this study with a key topic in Scientific Computing: **Quadrature**, which is also known as **Numerical Integration**.

Later, we'll use this as an opportunity to study the idea of **experimental analysis** of algorithms.

- ▶ A **Quadrature** method, in one dimension, is a method for estimating definite integrals. The applications are far too numerous to list, but feature in just about every area of Applied Mathematics, Probability Theory, and Engineering, and even some areas of pure mathematics.
- ▶ They are methods for estimating integrals of functions. So this gives us two reasons to code functions:
 - (i) As the functions we want to integrate;
 - (ii) As the algorithms for doing the integration.

But before we get on to actual methods, we'll learn the basics of writing functions in C++.

Functions

A good understanding of **functions**, and their uses, is of prime importance.

Some functions return/compute a single value. However, many important functions return more than one value, or modify one of its own arguments.

For that reason, we need to understand the difference between **call-by-value** and **call-by-reference** (← later).

Functions

Every C++ program has at least one function: `main()`

Example

```
#include <iostream>
int main(void )
{
    /* Stuff goes here */
    return(0);
}
```

Each function consists of two main parts: Header/Prototype and Body/Definition.

1. Header

The Function “header” or **prototype** gives the function's

- ▶ return value data type, or `void` if there is none, and
- ▶ parameter list data types or `void` if there are none.
- ▶ The header line ends with a semicolon.

The prototype is often given near the start of the file, before the **main()** section.

Syntax for function header:

```
ReturnType FnName (type1, type2, ...);
```


Examples:

int compute_lcm(int a, int b); // compute lowest common multiple of a and b

float convert_temp(float f); // convert between temp scales.

bool ComplicatedFunction(float f, double c, int x, int y, char ccc);

Other examples include passing a function as an argument.

but not

float, int ComputeTwoThings(int a, int b);

2. Function definition

- ▶ The **function definition** can be anywhere in the code (after the header). *(but not inside another function, such as main).*
- ▶ First line is the same as the prototype, except variables names need to be included, and that line does not end with a semi-colon.
- ▶ That is followed by the body of the function contained within curly brackets.

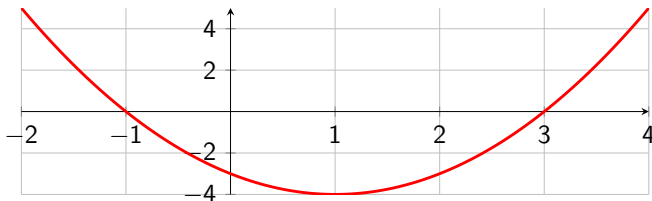
Syntax:

```
ReturnType FnName (type1 param1, type2 param2, ...)
{
    statements
}
```

- ▶ **ReturnType** is the data type of the data returned by the function.
- ▶ **FnName** the identifier by which the function is called.
- ▶ **type1 param1, ...** consists of
 - the data type of the parameter (*e.g., int, double, etc*)
 - the name of the parameter will have in the function. It acts within the function as a local variable. (*name=identifier*)
- ▶ the statements that form the function's body, contained with braces **{...}**.

Since this is a course on scientific computing, we'll often need to define mathematical functions from $\mathbb{R} \rightarrow \mathbb{R}$ (more or less), such as $f(x) = e^{-x}$. Typically, such functions map one or more **doubles** onto another **double**.

The example we'll look at is $f(x) = x^2 - 2x - 3$.



00MathFunction.cpp

```
1 #include <iostream>
2 #include <iomanip>
3
4 double f(double x) //  $x^2 - 2x - 3$ 
5 {
6     return (x*x - 2*x - 3);
7 }
8
9 int main(void){
10     double x;
11     std::cout << std::fixed << std::showpoint;
12     std::cout << std::setprecision(2);
13     for (int i=0; i<=10; i++)
14     {
15         x = -1.0 + i*.5;
16         std::cout << "f(" << x << ")=" << f(x) << std::endl;
17     }
18     return(0);
19 }
```

In this example, we write a function that takes an non-negative integer input and checks if it is a composite (`true`) or prime (`false`).

01IsComposite.cpp (header)

```

2 // 01IsComposite.cpp
  // An example of a simple function.

4 #include <iostream>

6 bool IsComposite(int i);
    
```

Calling the IsComposite function

01IsComposite.cpp (main)

```
8 int main(void )
  {
10     int i;

12     std::cout << "Enter a natural number: ";
    std::cin >> i;

16     std::cout << i << " is a " <<
        (IsComposite(i) ? "composite" : "prime") << " number."
        << std::endl;

    return(0);
20 }
```

Notice use of ?: operator.

Defining the IsComposite function

01IsComposite.cpp (function definition)

```
26 bool IsComposite(int i)
   {
   28     int k;
       for (k=2; k<i; k++)
           if ( (i%k) == 0)
   30         return(true);
   32     return(false); // If we get to here, i has no divisors between 2 and i-1
   }
```

Function ends first time a "return" is encountered.

Most functions will return some value. In rare situations, they don't, and so have a `void` return value.

02Kth.cpp (header)

```
2 // 02Kth.cpp:  
2 // Another example of a simple function.  
2 // Author: Niall Madden  
4 // Date: 05 Feb 2025  
4 // Week 04: CS319 - Scientific Computing  
6 #include <iostream>  
6 void Kth(int i);
```

Puzzle: What is the next term in the sequence:

s, t, n, d, r, d, t, h, t, h, t, h, ...



02Kth.cpp (main)

```
10 int main(void )  
   {  
       int i;  
  
       std::cout << "Enter a natural number: ";  
14       std::cin >> i;  
  
       std::cout << "That is the ";  
16       Kth(i);  
18       std::cout << " number." << std::endl;  
20       return(0);  
   }
```

*Sequence is 1st, 2nd, 3rd, 4th, 5th, 6th, 7th, 8th, 9th, 10th
11th, 12th, ... 19th, 20th,
21st, 22nd, ...*

02Kth.cpp (function definition)

```
24 // FUNCTION KTH
25 // ARGUMENT: single integer
26 // RETURN VALUE: void (does not return a value)
27 // WHAT: if input is 1, displays 1st, if input is 2, displays 2nd,
28 // etc.
29 void Kth(int i)
30 {
31     std::cout << i;
32     i = i%100;
33     if ( ((i%10) == 1) && (i != 11))
34         std::cout << "st";
35     else if ( ((i%10) == 2) && (i != 12))
36         std::cout << "nd";
37     else if ( ((i%10) == 3) && (i != 13))
38         std::cout << "rd";
39     else
40         std::cout << "th";
41 }
```

remainder on dividing by 100.

→ i = 1, 21, 31, ...

Numerical Integration

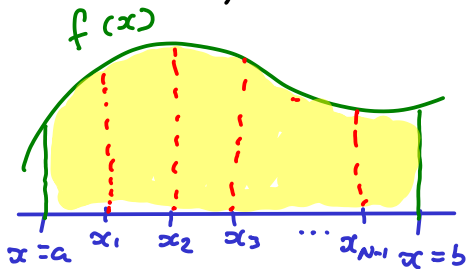
Numerical integration is an important topic in scientific computing. Although the history is ancient, it continues to be a hot topic of research, particularly when computing with high-dimensional data.

In this section, we want to estimate definite integrals of one-dimensional functions:

$$\int_a^b f(x) dx.$$

We'll use one of the simplest methods: the Trapezium Rule.

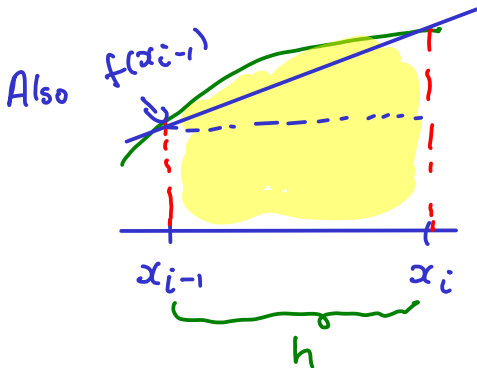
$\int_a^b f(x) dx$ is "area bounded above by $f(x)$, if $f > 0$, and the x -axis, as well as between $x=a$, $x=b$.



- Choose an int N .
- Set $h = \frac{(b-a)}{N}$
- Let $x_i = a + ih$ for $i = 0, 1, \dots, N$.

$$\begin{aligned} \text{So } x_0 &= a & \dots & x_N = b \\ x_1 &= a + h \\ x_2 &= a + 2h \end{aligned}$$

We know
$$\int_a^b f(x) dx = \int_{x_0}^{x_1} f(x) dx + \int_{x_1}^{x_2} f(x) dx + \dots + \int_{x_{N-1}}^{x_N} f(x) dx.$$



$$\begin{aligned} \int_{x_{i-1}}^{x_i} f(x) dx &\approx \\ &h f(x_{i-1}) \\ &+ \frac{h}{2} (f(x_i) - f(x_{i-1})) \\ &= \frac{h}{2} [f(x_{i-1}) + f(x_i)] \end{aligned}$$

Consequently

$$\int_a^b f(x) dx \approx \frac{h}{2} f(x_0) + h f(x_1) + h f(x_2) + \dots + \frac{h}{2} f(x_N).$$

03QuadratureV01.cpp (headers)

```
// 03QuadratureV01.cpp:
2 // Trapezium Rule (TR) quadrature for a 1D function
  // Author: Niall Madden
4 // Date: 06 Feb 2025
  // Week 04: CS319 - Scientific Computing
6 #include <iostream>
  #include <cmath>    // For exp()

double f(double); // prototype
10 double f(double x) { return(exp(x)); } // definition
```


03QuadratureV01.cpp (main)

```
12 int main(void )  
13 {  
14     std::cout << "Using the TR to integrate f(x)=exp(x)\n";  
15     std::cout << "Integrate f(x) between x=0 and x=1.\n";  
16     double a=0.0, b=1.0;  
17     double Int_f_true = exp(1)-1;  
18     std::cout << "Enter value of N for the Trap Rule: ";  
19     int N;  
20     std::cin >> N; // Lazy! Should do input checking.
```

03QuadratureV01.cpp (main continued)

```
22  double h=(b-a)/double(N);  
    double Int_f_TR = (h/2.0)*f(a);  
24  for (int i=1; i<N; i++)  
        Int_f_TR += h*f(a+i*h);  
26  Int_f_TR += (h/2.0)*f(b);  
  
28  double error = fabs(Int_f_true - Int_f_TR);  
  
30  std::cout << "N=" << N << ", Trap Rule=" << Int_f_TR  
        << ", error=" << error << std::endl;  
32  return(0);  
}
```

$x_i = a + ih$

Typical output:

Finished here

Wed

N	Error
2	3.565×10^{-2}
4	8.9×10^{-3}
8	2.2×10^{-3}
16	5.5×10^{-4}

It works!

Next it makes sense to write a function that implements the Trapezium Rule, so that it can be used in different settings.

The idea is pretty simple:

- ▶ As before, f will be a globally defined function.
- ▶ We write a function that takes as arguments a , b and N .
- ▶ The function implements the Trapezium Rule for these values, and the globally defined f .

04QuadratureV02.cpp (header)

```
2 // 04QuadratureV02.cpp: Trapezium Rule as a function
// Trapezium Rule (TR) quadrature for a 1D function
// Author: Niall Madden
4 // Date: Feb 2025
// Week 04: CS319 - Scientific Computing
6 #include <iostream>
#include <cmath> // For exp()
8 #include <iomanip>
10 double f(double x) { return(exp(x)); } // definition
double TrapRule(double a, double b, int N);
```

04QuadratureV02.cpp (main)

```
14 int main(void )
15 {
16     std::cout << "Using the TR to integrate in 1D\n";
17     std::cout << "Integrate between x=0 and x=1.\n";
18     double a=0.0, b=1.0;
19     double Int_true_f = exp(1)-1; // for f(x)=exp(x)
20
21     std::cout << "Enter value of N for the Trap Rule: ";
22     int N;
23     std::cin >> N; // Lazy! Should do input checking.
24
25     double Int_TR_f = TrapRule(a,b,N);
26     double error_f = fabs(Int_true_f - Int_TR_f);
27
28     std::cout << "N=" << std::setw(6) << N <<
29         ", Trap Rule=" << std::setprecision(6) <<
30         Int_TR_f << ", error=" << std::scientific <<
31         error_f << std::endl;
32     return(0);
```

04QuadratureV02.cpp (function)

```
34 double TrapRule(double a, double b, int N)
35 {
36     double h=(b-a)/double(N);
37     double QFn = (h/2.0)*f(a);
38     for (int i=1; i<N; i++)
39         QFn += h*f(a+i*h);
40     QFn += (h/2.0)*f(b);
41     return(QFn);
42 }
```

Functions as arguments to functions

We now have a function that implements the Trapezium Rule. However, it is rather limited, in several respects. This includes that the function, `f`, is hard-coded in the `TrapRule` function. If we want to change it, we'd edit the code, and recompile it.

Fortunately, it is relatively easy to give the name of one function as an argument to another.

The following example shows how it can be done.

Functions as arguments to functions

05QuadratureV03.cpp(header)

```
2 // 05QuadratureV03.cpp: Trapezium Rule as a function
// that takes a function as argument
// Week 04: CS319 - Scientific Computing
4 #include <iostream>
#include <cmath> // For exp()
6 #include <iomanip>

8 double f(double x) { return(exp(x)); } // definition
double g(double x) { return(6*x*x); } // definition

12 double TrapRule(double Fn(double), double a, double b,
int N);
```

Functions as arguments to functions

05QuadratureV03.cpp (part of main())

```
20  std::cout << "Which shall we integrate: \n"
    << "\t 1. f(x)=exp(x) \n\t 2. g(x)=6*x^2?\n";
22  int choice;
    std::cin >> choice;
24  while (!(choice == 1 || choice == 2) )
    {
26      std::cout << "You entered " << choice
          << ". Please enter 1 or 2: ";
28      std::cin >> choice;
    }
30  double Int_TR=-1; // good place-holder
    if (choice == 1)
32      Int_TR = TrapRule(f,a,b,10);
    else
34      Int_TR = TrapRule(g,a,b,10);

36  std::cout << "N=10" << ", Trap Rule="
    << std::setprecision(6) << Int_TR << std::endl;
38  return(0);
}
```

Functions as arguments to functions

05QuadratureV03.cpp (TrapRule())

```
42 double TrapRule(double Fn(double), double a,  
44                 double b, int N)  
44 {  
    double h=(b-a)/double(N);  
46    double QFn = (h/2.0)*Fn(a);  
    for (int i=1; i<N; i++)  
48        QFn += h*Fn(a+i*h);  
    QFn += (h/2.0)*Fn(b);  
  
    return(QFn);  
52 }
```

Functions with default arguments

In our previous example, we wrote a function with the header

```
double TrapRule(double Fn(double), double a, double b, int N);
```

And then we called it as

```
Int_TR = TrapRule(f,a,b,10);
```

That is, when we were not particularly interested in the value of N , we took it to be 10.

It is easy to adjust the function so that, for example, if we called the function as

```
Int_TR = TrapRule(f,a,b);
```

it would just be assumed that $N = 10$. All we have to do is adjust the function header.

Functions with default arguments

To do, this we specify the value of N in the **function prototype**. You can see this in `05QuadratureV04.cpp`. In particular, note Line 10:

`06QuadratureV04.cpp` (line 10)

```
10 double TrapRule(double Fn(double), double a,  
    double b, int N=10); // default N=10
```

This means that, if the user does not specify a value of N , then it is taken that $N = 10$.

Functions with default arguments

Important:

- ▶ You can specify default values for as many arguments as you like. For example:

```
1 double TrapRule(double Fn(double), double a=0.0,  
    double b=1.0, int N=10);
```

- ▶ If you specify a default value for an argument, you must specify it for any following arguments. For example, the following would cause an error.

```
2 double TrapRule(double Fn(double), double a=0.0,  
    double b=1.0, int N);
```

Pass-by-value

In C++ we need to distinguish between

- ▶ the value stored in the variable.
- ▶ a variable's identifier (might not be unique)
- ▶ a variable's (unique) memory address

In C++, if (say) `v` is a variable, then `&v` is the memory address of that variable.

We'll return to this at a later point, but for now we'll check the output of some lines of code that output a memory address.

07MemoryAddresses.cpp

```
10  int i=12;
    std::cout << "main: Value stored in i: " << i << '\n';
12  std::cout << "main: address of i: " << &i << '\n';
    Address(i);
    std::cout << "main: Value stored in i: " << i << '\n';
```

Typical output might be something like:

```
main: The value stored in i  is 12
```

```
main: The address of i is 0x7ffcd1338314
```


Pass-by-value

A while back we learned that, when we pass a variable as an argument to a function, a new **copy** of the variable is made.

This is called **pass-by-value**.

Even if the variable has the same name in both `main()` and the function called, and the same value, they are different: the variables are **local** to the function (or block) in which they are defined.

We'll test this by writing a function that

- ▶ Takes a `int` as input;
- ▶ Displays its value and its memory address;
- ▶ Changes the value;
- ▶ Displays the new value and its memory address.

07MemoryAddresses.cpp

```
18 void Address(int i)
   {
20     std::cout << "Address: Value stored in i: " << i << '\n';
       std::cout << "Address: address of i: " << &i << '\n';
22     i+=10; // Change value of i
       std::cout << "Address: New val stored in i: " << i << '\n';
24     std::cout << "Address: address of i: " << &i << '\n';
   }
```

Pass-by-value

Finally, let's call this function:

07MemoryAddresses.cpp

```
10  int i=12;
    std::cout << "main: Value stored in i: " << i << '\n';
    std::cout << "main: address of i: " << &i << '\n';
12  Address(i);
    std::cout << "main: Value stored in i: " << i << '\n';
14  std::cout << "main: address of i: " << &i << '\n';
```

Pass-by-value

In many case, “pass-by-value” is a good idea: a function can change the value of a variable passed to it, without changing the data of the calling function.

But sometimes we **want** a function to be able to change the value of a variable in the calling function.

The classic example is function that

- ▶ takes two **integer** inputs, **a** and **b**;
- ▶ after calling the function, the values of **a** and **b** are swapped.

Pass-by-value

08SwapByValue.cpp

```
4 #include <iostream>
   void Swap(int a, int b);

   int main(void )
8 {
    int a, b;

    std::cout << "Enter two integers: ";
12    std::cin >> a >> b;

14    std::cout << "Before Swap: a=" << a << ", b=" << b
        << std::endl;
16    Swap(a,b);
    std::cout << "After Swap: a=" << a << ", b=" << b
18        << std::endl;

20    return(0);
}
```

Pass-by-value

```
void Swap(int x, int y)
{
    int tmp;

    tmp=x;
    x=y;
    y=tmp;
}
```

This won't work.

We have passed only the *values stored in the variables a and b*. In the `swap` function these values are copied to local variables `x` and `y`. Although the local variables are swapped, they remained unchanged in the calling function.

What we really wanted to do here was to use **Pass-By-Reference** where we modify the contents of the memory space referred to by `a` and `b`. This is easily done...

Pass-by-value

...we just change the declaration and prototype from

```
void Swap(int x, int y) // Pass by value
```

to

```
void Swap(int &x, int &y) // Pass by Reference
```

the pass-by-reference is used.

Exercise (Simpson's Rule)

- ▶ Find the formula for Simpson's Rule for estimating $\int_a^b f(x)dx$.
- ▶ Write a function that implements it.
- ▶ Compare the Trapezium Rule and Simpson's Rule. Which appears more accurate for a given N ?

Exercise

Change the `Address()` function in `07MemoryAddresses.cpp` so that the variable `i` is passed by reference. How does the output change?